

CSE 719: Distributed & Cloud Computing

Lecture 3 — MapReduce & Hadoop

Model Answers: 2021, 2022 & 2024 Papers

University of Chittagong — 8th Sem B.Sc. (Engg.) — Source: Lecture-03 slides

Note: 2020 paper has no MapReduce questions. In 2024, all four parts of Q1 (Section A) are MapReduce — treat this as a core topic.

Contents

1	MapReduce Essentials (Reference)	2
2	2021 Examination (CSE 813, 52.5 marks)	2
2.1	Q7(c) — Various types of MapReduce jobs [2 marks]	2
3	2022 Examination (CSE 513, 54 marks)	3
3.1	Q4(a) — Two limitations on the Map function [2 marks]	3
3.2	Q4(b) — How MapReduce handles straggler tasks [2 marks]	4
4	2024 Examination (CSE-813, 54 marks)	4
4.1	Q1(a) — HPC fault tolerance using message passing [2.5 marks]	4
4.2	Q1(b) — MapReduce advantage over MPI for failure handling [2 marks]	5
4.3	Q1(c) — GFS replica location API: MapReduce optimization enabled [2.5 marks]	6
4.4	Q1(d) — Two limitations on the Map function (repeat) [2 marks]	6
5	Quick Summary — Exam Patterns & Must-Know	7

MapReduce Essentials (Reference)

MapReduce: A programming framework for **distributed and parallel processing on large datasets**. Consists of two phases: **Map** and **Reduce** (barrier between them: no Reduce starts until all Maps complete). Terms borrowed from functional programming (Lisp).

Execution pipeline:

Input (DFS) $\xrightarrow{\text{Map}}$ (key,value) pairs (local disk) $\xrightarrow{\text{Shuffle/Sort}}$ Reduce tasks (group by key) $\xrightarrow{\text{Reduce}}$ Output

Map: Called once per input record. Emits zero or more **intermediate (key, value)** pairs. Each Map task is **completely independent** of every other Map task.

Shuffle: Framework groups all intermediate values with the same key and routes them to one Reduce task. **Hash partitioning:** $\text{key} \rightarrow \text{reduce\#} = \text{hash}(\text{key}) \% R$ (number of reducers).

Reduce: Receives all values for one key; merges them into output. Each Reduce task operates independently.

Storage during execution:

- Map input: **from distributed FS** (GFS/HDFS).
- Map output: **to local disk** of the Map node.
- Reduce input: **pulled from local disks of Map nodes** (network read).
- Reduce output: **to distributed FS**.

WordCount (running example): Input <filename, "Welcome Everyone Hello Everyone"> $\xrightarrow{\text{Map}}$ (Welcome,1), (Everyone,1), (Hello,1), (Everyone,1) $\xrightarrow{\text{Reduce}}$ (Everyone,2), (Hello,1)

2021 Examination (CSE 813, 52.5 marks)

Q7(c) — Various types of MapReduce jobs

[2 marks]

What are the various types of MapReduce jobs?

The following are standard types of jobs that naturally fit the MapReduce model:

Job type	Map phase	Reduce phase
Word count	Emit (word, 1) for each word	Sum counts per word
Distributed grep	Emit line if it matches the pattern	Copy to output (identity)
Reverse web-link graph	For each link $a \rightarrow b$, emit (b, a)	Emit (b, list(a)) per target page
URL access frequency	Emit (URL, 1) per log entry	Sum per URL; chain 2 nd MR job for %
Distributed sort	Emit (value, _) identity	Identity; range partitioning across reducers
Inverted index	Emit (word, docID)	Emit (word, [docID list]) for search index

Common characteristics: large-scale data-intensive, embarrassingly parallel at the record level, naturally split into independent Map tasks. Particularly suited for **log processing, web search indexing, and ad-hoc queries.**

2022 Examination (CSE 513, 54 marks)

Q4(a) — Two limitations on the Map function

[2 marks]

What are the two important limitations MapReduce places on the Map function?

1. **No side effects / Must be a pure function.**

The Map function must be **deterministic**: given the same input record, it must always produce the same output. It cannot write to any **shared external state** (databases, counters, files) or communicate with other Map tasks during execution.

Why: Failed Map tasks are silently **re-executed** on another worker. If Map had side effects, re-execution would corrupt external state (e.g., a database counter would be incremented twice). Determinism also enables **speculative execution** (two copies of the same task; use whichever finishes first).

2. **No communication between Map tasks / Each record processed independently.**

A Map function sees **exactly one input record at a time**. It cannot read the outputs of other Map tasks, access sibling records, or maintain state across multiple Map invocations. Each Map task operates on its own input split in complete **isolation**.

Why: This independence is what makes MapReduce **trivially parallelis-**

able — all Map tasks can run simultaneously with no synchronisation overhead.

Q4(b) — How MapReduce handles straggler tasks [2 marks]

How does MapReduce handle straggler tasks?

Straggler: A task that runs significantly slower than its peers, delaying the entire job. Caused by: bad disk, network congestion, CPU throttling, or memory pressure on that node. Because of the **barrier** between Map and Reduce phases (no Reduce starts until all Maps finish), even one slow Map task stalls the whole job.

Solution: Speculative Execution (Backup Tasks)

1. The framework **tracks the progress** of each task (percentage complete).
2. When a job is **near completion** and some tasks are progressing abnormally slowly, the master launches **backup copies** of those straggler tasks on other available workers.
3. **Whichever instance finishes first** (original or backup) has its output accepted; the other is killed.
4. Result: tail latency is greatly reduced at the cost of a small increase in cluster resource usage (~a few percent extra tasks).

Key insight: Because Map functions are deterministic and have no side effects, running two copies of the same task is **always safe** — both will produce identical output.

2024 Examination (CSE-813, 54 marks)

Q1(a) — HPC fault tolerance using message passing [2.5 marks]

Describe the fault tolerance technique used in High Performance Computing (HPC) systems that use message passing.

Traditional HPC systems (using **MPI** — Message Passing Interface) rely on **Checkpoint/Restart** for fault tolerance:

1. **Periodic global checkpointing:** At regular intervals, **all** MPI processes synchronously **save their entire memory state** (variables, buffers, message queues) to stable storage (parallel file system).

2. **Restart on failure:** When any process fails, the **entire application** (all processes) rolls back to the last checkpoint and resumes from that saved state.
3. **Coordinated recovery:** All processes restart together to ensure a **consistent global state** (no dangling messages, no inconsistency between processes).

Limitations at scale:

- **Stop-the-world checkpointing** imposes regular pauses on all processes.
- **Checkpoint overhead** grows with application size; writing 100s of GB is slow.
- **Wasted work:** on failure, all processes restart even those that did not fail; all computation since the last checkpoint is lost.
- **Scalability crisis:** in a 10,000-node system where each node fails once per year, a failure occurs roughly every **53 minutes** — far shorter than a typical checkpoint interval, making the system permanently stalled.

Q1(b) — MapReduce advantage over MPI for failure handling [2 marks]

What advantage does MapReduce have over MPI-based systems for failure handling and recovery?

1. Task-level re-execution, not global restart.

When a MapReduce worker fails, only the **specific Map or Reduce tasks** running on that worker are re-scheduled on another available node. All other in-progress and completed tasks continue unaffected. Contrast with MPI: **all** processes restart from checkpoint.

2. No application-level checkpointing needed.

Map/Reduce tasks are **stateless and deterministic**: their output depends only on their input (read from the replicated distributed FS). Any failed task can be re-run from the original DFS input — which is still available because GFS/HDFS keeps **3 replicas**. There is no global state to save or restore.

3. Failure handling is transparent to the programmer.

The framework (master + YARN) detects failures via **heartbeats** and automatically re-queues tasks. The developer writes zero fault-tolerance code.

Summary: MPI requires saving and restoring the *entire global state*; MapReduce re-runs only the *individual failed tasks* — a fundamentally more scalable fault model.

Q1(c) — GFS replica location API: MapReduce optimization enabled [2.5 marks]

GFS exposes the locations of block replicas to clients via an API. What important MapReduce optimization does this enable?

The optimization enabled is **Data Locality** (also called *moving computation to data* or *locality-aware scheduling*).

How it works:

1. The MapReduce master **queries the GFS API** to learn which servers hold each input block's replicas.
2. It then **schedules each Map task preferentially** on:
 1. A server that already holds a replica of that block (**local disk read**), or
 2. A server on the **same rack** as one holding a replica (intra-rack read), or
 3. Any server anywhere (fallback — full cross-rack network read).
3. Most Map tasks read their input data **from local disk** instead of over the network.

Why it matters:

- Eliminates network bandwidth as a bottleneck for the Map phase (which reads the entire input dataset).
- In a large cluster, **network cross-section bandwidth** is the scarcest resource; local reads conserve it for the shuffle phase (which must traverse the network).
- Measured speedup: significantly faster job completion at scale (often the dominant optimisation in MapReduce performance).

Q1(d) — Two limitations on the Map function (repeat) [2 marks]

What are two important limitations that MapReduce places on the Map function?

(Same question as 2022 Q4(a) — see full answer there.)

Summary (two limitations):

1. **No side effects — must be a pure/deterministic function.** Map must produce the same output for the same input; it cannot write to shared external state. This is required for safe re-execution on failure and for speculative execution.
2. **No inter-task communication — each record processed independently.** A Map function sees only one input record at a time; it cannot access other records or the outputs of sibling Map tasks. This is what makes MapReduce trivially parallelisable.

Quick Summary — Exam Patterns & Must-Know

- **2024 entire Q1 = MapReduce** (4 parts, 9 marks of Section A). Trend: MapReduce yield has jumped from 3/5 to 4/5. Treat it as a primary topic.
- **Two Map limitations (2022 Q4a = 2024 Q1d, verbatim repeat):** (1) No side effects / deterministic. (2) No inter-task communication / independent records. Know both with the WHY (re-execution + parallelism).
- **Straggler / Speculative Execution (2022 Q4b):** Define straggler → barrier problem → track progress % → launch backup copy → use whichever finishes first.
- **MPI vs. MapReduce fault tolerance (2024 Q1a + Q1b):** MPI = global checkpoint/restart (all processes stop and roll back). MapReduce = re-run only failed task (task-level, stateless, DFS replicas survive node death).
- **GFS locality (2024 Q1c):** GFS API reveals replica locations → master schedules Map tasks co-located with data → eliminates network reads in Map phase → major performance gain. Priority: same node → same rack → anywhere.
- **Types of MapReduce jobs (2021 Q7c):** Word count, distributed grep, reverse web-link graph, URL frequency, distributed sort, inverted index. Know Map/Reduce steps for at least word count and grep cold.
- **YARN (not directly examined yet, but know components):** Resource Manager (global scheduler) + Node Manager (per-server daemon) + Application Master (per-job; detects task failures).
- **Key pipeline to draw:** Input (DFS) → Map (local disk output) → Shuffle/hash-partition → Reduce (DFS output). Barrier: all Maps must finish before Reduce starts.